Дисциплина: Языки и технология программирования · Лекции 1–8

Система бронирования переговорных

Предметная область: офис. Цель: исключить пересечения встреч и упростить учёт занятости комнат. Язык реализации: Python, консольное приложение.

Выполнил: Жаксылык Жаркын, группа ИС25-21

Пользователи и сценарии

Сотрудник

Просматривает занятость комнат на выбранный день. Создаёт бронь на своё имя, указывая комнату, начало и конец встречи. Отменяет только свои брони, если планы изменились.

Администратор

Видит все брони по всем комнатам. Может отменить любую бронь, например при переносе мероприятия. Следит за загрузкой и добавляет новые переговорные.

Основные сценарии

1. Бронирование: ввод данных, проверка, подтверждение с номером брони.
2. Отказ: при пересечении или нарушении правил программа объясняет причину.
3. Отмена: по номеру брони, после чего слот снова доступен.
4. Просмотр: список занятости комнаты, отсортированный по времени.

Подход к разработке и ограничения

Жизненный цикл ПО

Выбрана итерационная модель, так как требования уточняются по ходу работы. Этапы: анализ требований, проектирование модели данных, кодирование, тестирование, демонстрация. Итерация 1: ядро (создание и отмена). Итерация 2: проверки и ограничения. Итерация 3: меню и вывод.

Ограничения бронирования

Рабочее время 09:00–18:00. Длительность от 30 минут до 4 часов, шаг 30 минут. Бронь только на будущее время. В одной комнате встречи не пересекаются, но конец одной может совпадать с началом другой. Вместимость комнаты не меньше числа участников.

Почему итерационная модель, а не каскадная: каждая итерация даёт работающую версию, которую можно проверить тестами сразу, а ошибки находятся на ранних этапах.

Модель данных

Сущность	Поля	Назначение
Room (dataclass)	name, capacity	Переговорная: название и вместимость
Booking (dataclass)	id, room, user, start, end, status	Одна бронь с временем и владельцем
Status (Enum)	ACTIVE, CANCELLED	Состояние брони, отмена без удаления
Списки	rooms, bookings	Хранение данных в памяти программы

Отмена меняет статус, а не удаляет запись, поэтому история сохраняется. Время хранится как datetime, что упрощает сравнение интервалов. Применены структуры и перечисления (лекция 8) и коллекции (лекция 6).

Функциональные возможности

Функция	Что делает	Тема
create_booking()	Проверяет данные и создаёт бронь	Функции, строки
cancel_booking(id)	Находит бронь и ставит статус CANCELLED	Циклы, enum
is_free(room, start, end)	Перебирает брони и ищет пересечение	Циклы
list_schedule(room)	Выводит занятость, сортируя по времени	Массивы
find_free_rooms(start, end)	Подбирает свободные комнаты на интервал	Коллекции

Главная логика сосредоточена в is_free(): её используют создание брони и поиск свободных комнат, поэтому правило пересечения записано в одном месте.

Проверка ввода

```
def parse_time(text):
  try: return datetime.strptime(text.strip(), "%Y-%m-%d %H:%M")
  except ValueError: raise BookingError("Формат: ГГГГ-ММ-ДД ЧЧ:ММ")
```

- Имя пользователя не пустое, лишние пробелы удаляются методом strip().
- Комната должна существовать в списке rooms, иначе сообщение об ошибке.
- Формат даты проверяется при разборе строки, ошибка не завершает программу.
- Начало раньше конца, минуты кратны 30, интервал внутри 09:00–18:00.
- Пересечение: **start < b.end and end > b.start** для активных броней.

Тестовые сценарии

№	Вход	Ожидаемый результат
1	Свободная комната, 10:00–11:00	Бронь создана
2	Та же комната, 10:30–11:30	Отказ: пересечение
3	Начало 11:00 при конце другой брони 11:00	Бронь создана
4	Дата «завтра в 5»	Ошибка формата
5	Начало позже конца	Ошибка интервала
6	Время 20:00–21:00	Отказ: вне рабочих часов
7	Отмена брони и повторная бронь слота	Слот свободен, бронь создана
8	Отмена несуществующего номера	Сообщение: бронь не найдена

Демонстрация и выводы

```
> book Alfa "Иван" "2026-10-16 10:00" "2026-10-16 11:00"
Бронь №1 создана
> book Alfa "Анна" "2026-10-16 10:30" "2026-10-16 11:30"
Ошибка: комната занята
> cancel 1
Бронь №1 отменена
```

- Реализованы создание и отмена брони, проверка времени, список занятости и поиск свободных комнат.
- Применены циклы, функции, коллекции, обработка строк, структуры и перечисления.
- Развитие: сохранение в файл JSON, повторяющиеся брони, уведомления.